The Open Web Application Security Project (OWASP)

The Open Web Application Security Project (OWASP) (*http://owasp.org*) is a not for profit community dedicated to help developers design and develop secure Web applications. It enables them to prioritise their efforts by publishing various standards, guides, cheat sheets, etc, in order to secure applications. Started in 2001 by Mark Curphey, it has grown to include several organisations, educational institutions and volunteers working towards building a secure Web.

Among its various projects, every three years, OWASP publishes a list of the top 10 vulnerabilities that plague Web applications. The list is published after extensive research, which includes the survey of the top four consulting companies in the world and three software vendors. In total, the project goes through a database of approximately 500,000 vulnerabilities and shortlists the top 10. The list was last published in 2013.

In addition to publishing this list, OWASP also includes the means and methods to counter them. The following section briefly describes each of the 10 vulnerabilities and their countermeasures (for more details, please visit *http://goo.gl/p6rAzr).*

OWASP's top 10 vulnerabilities, 2013

1. Injection

A Web application is vulnerable to injection when it accepts commands or queries in input fields, meant for obtaining information from the user. A classic example is SQL Injection, wherein the attacker injects SQL queries in an input field in order to bypass the authentication mechanism.

How to fix it: Use safe or parametrised APIs, escape special characters, provide white list input validation (e.g., if a field is meant to accept numeric values, the application should not permit the user to enter letters of the alphabet).

2. Broken authentication and session management

This flaw can occur when authentication mechanisms are not implemented properly (e.g., sending or storing credentials in plain text; when password recovery allows passwords to be changed without proper authentication and verification of the user, etc) or when sessions are poorly managed (e.g., if the session time-out is not defined; if session IDs are exposed in the URL, etc).

How to fix it: Use standards like ISO 27001:2013 or OWASP's Application Security Verification Standard (ASVS) when defining authentication and session management requirements for the Web application. Ensure that the Cross Site Scripting (XSS - explained later) flaw is taken care of.

3. Cross Site Scripting (XSS)

A Cross Site Scripting vulnerability allows an attacker to insert malicious script/code in a Web page that directs the user's browser to perform a malicious action. For example, an attacker can leave a simple JavaScript code in a website's comments section, which redirects anyone visiting that website to a fake login page.

How to fix it: Avoid special characters, use white list input validation, use auto-sanitisation libraries for rich content, and employ the Content Security Policy to protect the entire website from XSS.

4. Insecure direct object references

Web applications often expose actual object names to users while generating Web pages (e.g., <URL>/ acntID='1234'). However, if they fail to verify a user's access privilege to that particular object, users could manipulate the object value and access information that they are not authorised to. For example, in the above URL, a user may put acntID as '4567' and be able to access the information of that account even though he's not privileged to do so.

How to fix it: Check users' access privileges to each object reference before granting access to it; use session-specific or user-specific mapping of objects to avoid direct references.

5. Security misconfiguration

Most software like Web servers, database servers, programming platforms, etc, ship with disabled security controls. Often, Web developers either forget to configure